# Flutter

Séance 1 · Pourquoi Flutter, installation, Dart

# Télécharger le SDK Flutter 3.47.8

L'archive fait **1,6 Go**.

`docs.flutter.dev` › Install › Archive › Stable channel › Linux › **3.47.8**

Le cours de la prochaine heure se suit sans machine.

# Une application construite sur 12 séances

- **12 séances de 4 h**
- **Pantry**, gestion de garde-manger, sur l'API Open Food Facts
- Cible **Chrome** jusqu'à la séance 7, **Android** à partir de la séance 8
- Structure d'une séance de cours : deux blocs de cours, deux TP, un débrief

## Évaluation

- **80 %** : rendu final en équipe, travaillé en séances 11 et 12, présenté en séance 6
- **20 %** : contrôle continu sur les livrables de TP, présenté en séance 2

# Hors du cours : stores, backend, iOS

- **Stores, signature, CI/CD** : survolés en séance 10
- **Backend** : les API sont consommées, pas écrites
- **iOS et desktop** : non testés dans le cours, libre à chacun d'essayer

# Aujourd'hui

| Horaire | Bloc |
|---|---|
| **0:00 – 0:20** | Lancement des téléchargements |
| **0:20 – 1:05** | Pourquoi Flutter |
| **1:05 – 2:00** | Installation en autonomie |
| **2:00 – 2:15** | Pause |
| **2:15 – 3:15** | Dart en accéléré, sur DartPad |
| **3:15 – 4:10** | Codelab *Your first Flutter app* |
| **4:10 – 4:15** | Consignes |

Objectif unique de la séance : `flutter run -d chrome` **fonctionne**.

# Pourquoi Flutter

Le paysage, l'architecture, les limites

# Quatre façons de faire une application mobile

| Approche | Langage | Rendu |
|---|---|---|
| **Natif** | Kotlin / Swift | composants du système |
| **React Native** | TypeScript | composants du système, pilotés depuis JS |
| **KMP + Compose Multiplatform** | Kotlin | Compose : natif sur Android, Skia ailleurs |
| **Flutter** | Dart | moteur de rendu propre |

# Natif : deux bases de code

- **Avantages** : nouveautés du système dès leur sortie, outillage mature, performances de référence
- **Limites** : chaque fonctionnalité écrite deux fois, par deux équipes
- **Adapté** : une seule plateforme, ou une forte intégration au système

# React Native : du JavaScript qui pilote le natif

- **Avantages** : écosystème JavaScript, compétences web réutilisées, rendu natif
- **Limites** : la frontière JavaScript ↔ natif coûte selon l'usage, bibliothèques de qualité inégale

# KMP : partager la logique, garder l'interface

**Kotlin Multiplatform** partage la logique métier ; l'interface reste native, ou devient partagée avec **Compose Multiplatform**.

- **Avantages** : adoption progressive, Kotlin et Compose déjà connus
- **Limites** : moins de bibliothèques d'interface que Flutter, interface iOS stable depuis 2025 seulement

> Alternative la plus proche de Flutter pour qui vient de Compose.

# Flutter dessine chaque pixel

Flutter ne demande pas au système de créer un bouton : il le dessine lui-même, avec son moteur de rendu (**Impeller** sur mobile, **CanvasKit** dans le navigateur).

- Code Dart compilé en **AOT** en production
- Une seule base de code : Android, iOS, web, Windows, macOS, Linux

# Flutter reçoit une surface vide et y dessine tout

1. Le système fournit une **surface vide** : une fenêtre sur mobile, un `<canvas>` dans le navigateur
2. Flutter calcule la taille et la position de chaque widget
3. Le moteur **dessine** textes, boutons, ombres et curseurs dans cette surface, à chaque image

Dans Chrome, l'inspecteur d'une application Flutter ne montre qu'un `<canvas>`, sans aucun `<button>`.

# Avantages de Flutter

- **Même rendu sur tous les appareils**, quelle que soit la surcouche du constructeur
- **Mêmes composants sur un Android ancien ou récent** : ils viennent de Flutter, pas du système
- **Hot reload** : le code change, l'état de l'application reste
- **Tout est widget** : mise en page, marge, style, animation

# Limites de Flutter

- **Binaire plus lourd** : le moteur est embarqué, même dans une application vide
- **Accès au système par plugin** : caméra, notifications, etc., vus en séance 8
- **Apparence iOS à choisir soi-même** : Material par défaut, widgets Cupertino et `.adaptive` en séance 9
- **Un seul éditeur**, Google, décide de la feuille de route
- **Paquets tiers inégalement maintenus** : un tutoriel de plus de 18 mois est souvent obsolète

# Les quatre approches répondent à des contraintes différentes

Le choix de Flutter pour ce cours n'est pas un classement.

# La stack du cours

| Domaine | Choix | Côté Android |
|---|---|---|
| État | `setState` puis **`flutter_bloc`** | `ViewModel` + `StateFlow` |
| Injection | **`get_it`** puis **`injectable`** | Koin, Hilt |
| Navigation | `Navigator` puis **`go_router`** | Navigation Compose |
| Réseau | **`dio`** | Retrofit, OkHttp |
| Modèles | **`freezed`** + `json_serializable` | `data class` + kotlinx.serialization |
| Base locale | **`drift`** | Room |

# Installation

55 minutes · objectif `flutter run -d chrome`

# Chrome est la cible jusqu'à la séance 8

`flutter run -d chrome` ne demande ni SDK Android, ni Gradle, ni émulateur, ni droits administrateur : c'est la cible la plus simple à installer.

- Le cours et les TP ciblent Chrome jusqu'à la séance 8
- Android ou une autre plateforme reste possible en parallèle dès la séance 2, installée de son côté

# Installer Flutter 3.47.8

1. Prendre l'archive **3.47.8** téléchargée en début de séance et suivre **docs.flutter.dev/install/manual**
2. Lancer `flutter doctor`
3. Pour la suite de la séance, **seule la ligne Chrome doit être verte**

Ne pas lancer `flutter upgrade` : tout le cours repose sur la version 3.47.8. Le détail est dans `docs/seances/01/installation.md`, sur le dépôt du cours.

# Pause

15 minutes

# Dart

Écarts avec Kotlin

# Dart en quatre temps sur DartPad

- Quatre explications courtes, chacune suivie d'exercices sur **dartpad.dev**, sans installation
- Énoncés : `docs/seances/01/exercices-dart.md` · correspondances Kotlin ↔ Dart : `docs/seances/01/dart-pour-devs-kotlin.md`
- Exercices non terminés : à reprendre pendant le codelab

# Les paramètres nommés d'un constructeur de widget

```
class Tile extends StatelessWidget {
  const Tile({super.key, required this.title, this.quantity = 1});

  final String title;
  final int quantity;

  @override
  Widget build(BuildContext context) => Text('$title × $quantity');
}

Tile(title: 'Pasta', quantity: 3)
```

- `{ }` : paramètres nommés
- `required` : obligatoire · `= 1` : valeur par défaut
- `this.title` : affecte le champ · `super.key` : transmis au parent
- Type avant le nom : `int quantity`, là où Kotlin écrit `quantity: Int`

# Null safety : presque comme Kotlin

| Kotlin | Dart |
|---|---|
| `String?` | `String?` |
| `x!!` | `x!` · un seul point d'exclamation |
| `x ?: y` | `x ?? y` |
| `lateinit var` | `late` · y compris sur un `final` |

```dart
late final String value = expensiveComputation();  // evaluated on first access
```

Avec un initialiseur, `late` devient lazy, ce que `lateinit` ne permet pas.

# Un widget `const` n'est pas reconstruit par son parent

| Mot-clé | Signification |
|---|---|
| `final` | assigné une fois, **valeur calculée à l'exécution** |
| `const` | valeur connue **à la compilation** |

```
final now = DateTime.now();          // ok
const pi = 3.14159;                   // ok
```

- Deux `const` identiques sont **le même objet** : `identical(const [1, 2], const [1, 2])` renvoie `true`
- Un widget `const` reste donc la même instance à chaque build du parent, qui ne le reconstruit pas (séance 2)

# Exercice 1

null safety, 5 minutes

`docs/seances/01/exercices-dart.md`

# Un `await` oublié ne lève aucune erreur

| Kotlin | Dart |
|---|---|
| `suspend fun` | `Future<T>` + `async` |
| `Flow<T>` | `Stream<T>` |
| `collect { }` | `await for` |
| structured concurrency | **rien d'équivalent** |

```dart
Future<String> load() async {
  await Future.delayed(const Duration(seconds: 1));
  return 'loaded';
}
```

- Une fonction `async` renvoie **toujours** un `Future`
- Un `await` oublié ne provoque aucune erreur, ni à la compilation ni à l'exécution
- Les lints `unawaited_futures` et `discarded_futures` signalent cet oubli

# Tout le code Dart tourne sur un seul thread

- `await` rend la main à l'event loop : rien ne part en arrière-plan
- Un calcul long **gèle l'interface**, même dans une fonction `async`

```dart
print('1');
Future(() => print('2'));
Future.microtask(() => print('3'));
print('4');
```

Affiche `1 4 3 2` : le code synchrone d'abord, puis les microtasks, puis les événements.

# Exercices 2, 3 et 4

Event loop · `Future` · `Stream`, 15 minutes

Sur l'exercice 3 : retirer le `await`, et regarder ce qui s'affiche.

# Une liste se construit avec `if`, `for` et spread

```dart
final widgets = [
  const Header(),
  if (showTitle) const Heading(),
  ...otherWidgets,
  for (final p in products) ProductRow(p),
];
```

Remplace les `if` et les boucles dans la construction d'une interface, utilisé dès la séance 2.

`map` et `where` sont **lazy** : rien n'est calculé sans `.toList()`.

```dart
final names = products.map((p) => p.name);           // Iterable: nothing computed yet
final list = products.map((p) => p.name).toList();   // List: computed now
```

# Exercice 5

collection-if et spread, 5 minutes

# Le compilateur vérifie chaque cas

```
sealed class Result {}
class Success extends Result { Success(this.data); final String data; }
class Failure extends Result { Failure(this.message); final String message; }

String describe(Result r) => switch (r) {
  Success(:final data)    => 'ok: $data',
  Failure(:final message) => 'error: $message',
};
```

- `Success(:final data)` : vérifie que `r` est un `Success` et place son champ `data` dans une variable `data`
- La compilation échoue si un cas manque
- Sert dès la séance 4 pour les quatre états d'un écran : chargement, données, erreur, vide

# Exercice 6

`sealed` et `switch` exhaustif, 10 minutes

Une fois qu'il fonctionne : retirer un cas du `switch`, et lire l'erreur.

# Absent de Dart

| Kotlin | Dart |
|---|---|
| `let`, `apply`, `run`, `also` | la cascade `..`, partiellement |
| `data class` | classe ordinaire, ou le paquet `freezed` (séance 4) |
| `object` / `companion object` | variable de premier niveau, ou `static` |
| coroutines, structured concurrency | `Future` / `Stream`, sans annulation automatique |
| `private`, `internal` | préfixe `_` : privé **au fichier**, pas à la classe |

# Codelab : *Your first Flutter app*

55 minutes · `codelabs.developers.google.com` → *Your first Flutter app*

En autonomie, sur **Chrome**, où le hot reload fonctionne.

# Avant la séance 2

- Terminer l'installation : `flutter run -d chrome` doit fonctionner
- Parcourir le catalogue de widgets et quelques *Widget of the Week*
- Terminer le codelab

Séance 2 : widgets, mise en page, état local, formulaires. Début du développement de Pantry.

# Questions

# Pointeurs

| Ressource | Adresse |
|---|---|
| **Dépôt du cours** | `github.com/mduruisseau/pantry-2026` → `docs/seances/01/` |
| Installation | `docs.flutter.dev/install/manual` → Linux |
| Archive 3.47.8 | `docs.flutter.dev/install/archive` → Stable · Linux · 3.47.8 |
| Codelab | `codelabs.developers.google.com/codelabs/flutter-codelab-first` |
| DartPad | `dartpad.dev` |
| Langage Dart | `dart.dev/language` |
| Catalogue de widgets | `docs.flutter.dev/ui/widgets` |
| Cookbook et ressources | `docs.flutter.dev/reference/learning-resources` |
| Samples | `flutter.github.io/samples` |
| Widget of the Week | chaîne Flutter sur YouTube |
| Référence de l'API | `api.flutter.dev` |